

Proyecto Final: Crash Surfing

Momento II

Marcos Castrillón Velásquez
Ingeniería Electrónica
Universidad de Antioquia
Medellín, Colombia
marcos.castrillon@udea.edu.co

Luis Andrés Castillo Chicaiza
Ingeniería Electrónica
Universidad de Antioquia
Medellín, Colombia
luis.castillo1@udea.edu.co

I. ACTUALIZACIÓN DEL CONTEXTO Y DETALLES ADICIONALES

Se han realizado algunos ajustes en la lógica de juego y los modelos físicos para asegurar la viabilidad en Qt.

A. Sistema de Vidas y Progresión

Crash iniciará cada nivel con cinco (5) vidas. Una colisión con un obstáculo (sierras, cajas TNT) resultará en la pérdida de una vida. La recolección de frutas Wumpa no restaurará vidas, pero sumará puntaje. Al llegar a cero vidas, se activará el estado de "Game Over". La progresión se visualizará mediante una barra de estado que mide la distancia recorrida en el Nivel 1 o la recolección de cristales en el Nivel 2.

B. Modificación de Modelos Físicos en Nivel 1

Se ha eliminado el modelo de movimiento ondulatorio para la superficie del agua. El nivel se desarrollará sobre una superficie plana estática para simplificar la detección de colisiones de base.

C. Simplificación Dinámica en Nivel 2

Se eliminó la implementación de vectores de corriente variable para el flujo del agua en el remolino. El control se hará solamente con fuerza centrípeta constante para la succión y desplazamiento del Jetboard, evitando la complejidad de cálculos de campos vectoriales en cada frame.

II. DESCRIPCIÓN DETALLADA: NIVEL 1 (EL RÍO CHIGORODÓ)

A. Vistas y Entorno

El nivel se implementa utilizando una **vista lateral (Side-scroller)**. La cámara presenta un desplazamiento automático (*scroll*) hacia la derecha a una velocidad base v_{scroll} constante. El eje Horizontal (X) representa la progresión de la barra de mapa, y el eje Vertical (Y) representa la altura de Crash y los obstáculos. El fondo de la escena será estático o con efecto parallax.

B. Personajes e Interacciones

- **Crash Bandicoot (Jugador):** Su posición en X es relativa a la cámara. El jugador controla exclusivamente el movimiento en el eje Y. Se implementa una interacción de salto parabólico al presionar una tecla definida. No existe aceleración horizontal manual; la progresión depende del *scroll* automático.
- **Obstáculos Dinámicos (Péndulos/Sierras):** Estos objetos se mueven de forma autónoma describiendo oscilaciones horizontales del río.
- **Obstáculos Estáticos:** Troncos y piedras fijas en el agua.
- **Ítems (Wumpa):** Frutas estáticas distribuidas a distintas alturas que se destruyen y suman puntaje al contacto.

C. Modelos Físicos

- 1) **Tiro Parabólico (Salto):** Al activar el salto, se aplica una velocidad inicial vertical v_{0y} . La posición vertical $y(t)$ se calcula frame a frame dentro del *QTimer* principal utilizando la ecuación de cinemática:

$$y(t) = y_0 + v_{0y}t - \frac{1}{2}gt^2 \quad (1)$$

Donde g es una constante de gravedad ajustada para el entorno 2D. La colisión con la superficie del agua o una plataforma detiene este modelo.

- 2) **Movimiento Oscilatorio Armónico (MOA):** Para los péndulos o sierras que oscilan horizontalmente, su coordenada x respecto a su punto de anclaje se calcula mediante:

$$x(t) = A \cdot \cos(\omega t + \phi) \quad (2)$$

Donde A es la amplitud de la oscilación y ω la frecuencia angular. Este movimiento se actualiza en el *QTimer* de cada objeto.

- 3) **Detección de Colisiones y Transferencia de Momento:** Se utilizará o implementará una función de colisión de Qt. Ante estas, con cajas TNT o sierras, se reduce una vida y se activa una rutina de invencibilidad temporal (parpadeo del sprite). Para cajas de madera Wumpa, se aplica una disminución de velocidad simulando el impacto al destruir la caja.

III. DESCRIPCIÓN DETALLADA: NIVEL 2 (EL VÓRTICE DIMENSIONAL)

A. Vistas y Entorno

Este nivel adopta una **vista cenital (Top-down)**, visualizando el escenario circular del remolino desde arriba. Las coordenadas (X, Y) representan el plano de la superficie del océano. La escena es una arena fija sin *scroll* de cámara. El límite exterior de la arena representa la zona segura inicial, y el centro (x_c, y_c) representa el centro del remolino.

B. Personajes e Interacciones

- **Crash Bandicoot (Jugador):** Tiene libertad de movimiento en 360 grados (X e Y). El jugador controla la aceleración y dirección del Jetboard. Crash debe navegar de modo que contrarreste la succión central.
- **El Remolino:** Aplica una fuerza centrípeta de succión constante hacia el centro sobre cualquier objeto en la escena
- **Cajas de Daño (TNT):** Obstáculos estáticos o que giran lentamente alrededor del centro, los cuales aplican daño al contacto.
- **Cristales de Energía (Objetivos):** Cinco ítems estáticos distribuidos radialmente cerca de zonas de alto peligro que deben ser recolectados para completar el nivel.

C. Modelos Físicos

- 1) **Fuerza Centrípeta (Succión del Remolino):** En cada actualización del temporizador ('QTimer'), se calcula un vector de aceleración $\vec{a}_c$ que apunta desde la posición actual del jugador $\vec{P}(x, y)$ hacia el centro del remolino $\vec{C}(x_c, y_c)$. La magnitud de esta aceleración radial a_c es constante o puede aumentar linealmente a medida que la distancia r al centro disminuye:

$$\vec{F}_c = -\frac{v^2}{r}\hat{r} \implies \vec{a}_c = k \cdot (\vec{C} - \vec{P}) \quad (3)$$

Donde k es un factor de succión parametrizable. Esta aceleración se suma al vector de velocidad del jugador.

- 2) **Inercia de Deslizamiento (Movimiento sobre agua):** El Jetboard conserva su cantidad de movimiento. Al soltar las teclas de dirección, la velocidad del personaje no llega a cero instantáneamente. Se aplica un factor de amortiguación constante μ en cada frame:

$$\vec{v}(t+1) = \vec{v}(t) \cdot (1 - \mu) \quad (4)$$

Donde $0 < \mu < 1$. Esto simula el deslizamiento sobre el agua.

- 3) **Dinámica de Explosiones y Impacto:** Al colisionar con cajas TNT, se genera un vector de fuerza repulsivo instantáneo $\vec{F}_{exp}$ con origen en el centro de la explosión. Este vector se aplica sobre el jugador, empujándolo y restando una vida.

IV. SPRITES

A continuación se presetan los sprites iniciales para el nivel 1 (Fig. 1) y el nivel 2 (Fig. 2).



Fig. 1. Sprites nivel 1



Fig. 2. Sprites nivel 2

V. DIAGRAMA DE CLASES

El diagrama de clases propuesto para el desarrollo del juego (Fig. 3). Este diagrama describe la arquitectura lógica del sistema, donde la clase *Game* actúa como núcleo encargándose de la gestión del ciclo de actualización, las entidades y la interacción entre componentes. Se emplea herencia a través de la clase *Entity*, permitiendo modelar de forma flexible al jugador, los enemigos, los obstáculos y los ítems. Además, se incorpora el patrón *Strategy* mediante la clase *Agent*, asociada a los enemigos, lo cual permite definir distintos comportamientos inteligentes sin acoplar la lógica directamente a las entidades.

VI. ARQUITECTURA DEL AGENTE INTELIGENTE

A continuación, se presenta el diagrama de flujo del agente inteligente (Fig. 4). Este flujo describe el ciclo de funcionamiento del agente basado en cuatro componentes principales: percepción, decisión, acción y aprendizaje. En cada iteración, el agente obtiene información del entorno, procesa estos datos para tomar una decisión, ejecuta una acción sobre el juego y finalmente ajusta su comportamiento mediante un mecanismo de aprendizaje simple.

VII. GITHUB

Link de GitHub: <https://github.com/LuisC645/ProyectoFinal>

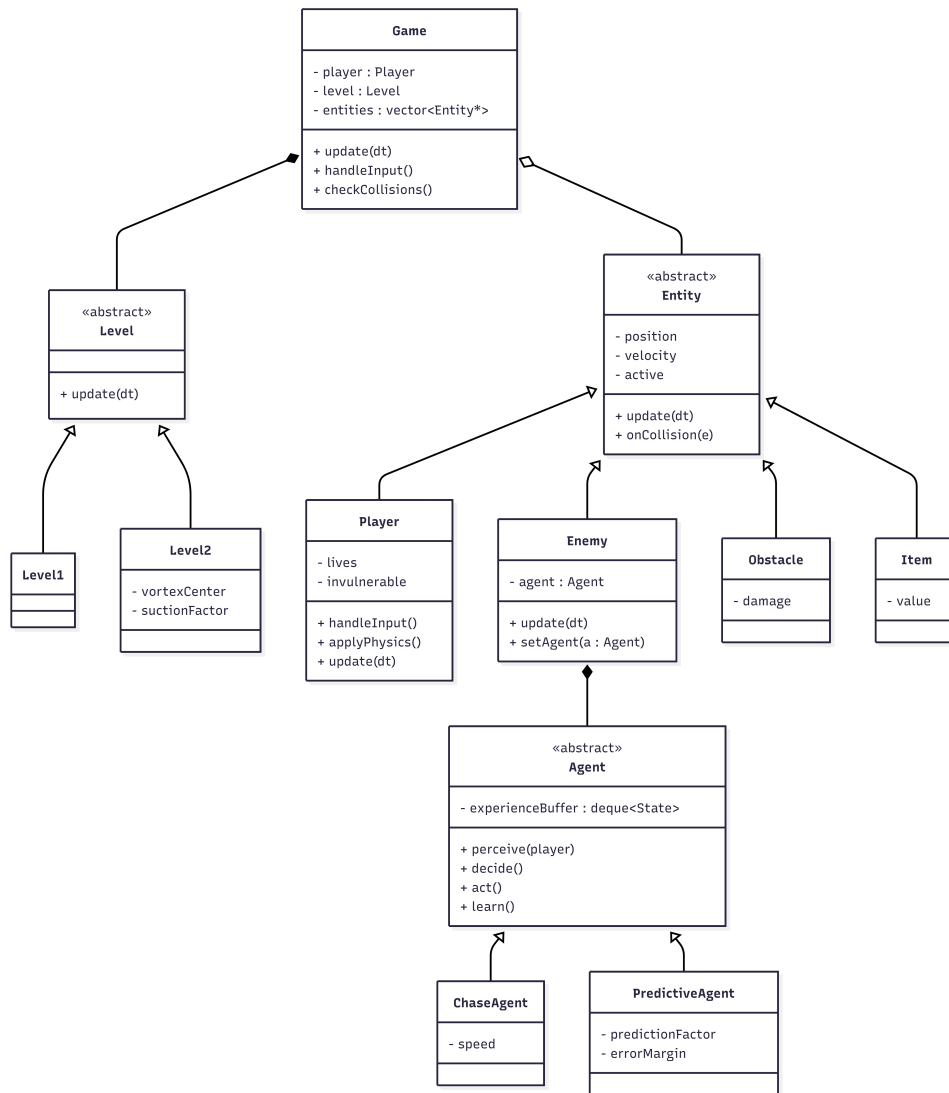


Fig. 3. Diagrama de clases del sistema

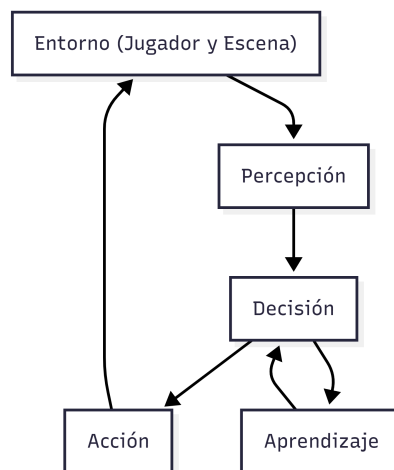


Fig. 4. Diagrama de flujo del agente inteligente